

ere's what I will include in the PDF:

1. Project Structure

```
/var/www/
├── myflaskapp/
│   ├── venv/           # Python virtual environment
│   ├── testserver1.py  # Your Flask app
│   └── editor.json      # Firebase credentials
```

2. Python Virtual Environment

```
cd /var/www/myflaskapp
python3 -m venv venv
source venv/bin/activate
pip install flask flask-cors google-cloud-storage google-cloud-firestore
PyPDF2 pandas bcrypt gunicorn
```

3. Gunicorn Systemd Service (/etc/systemd/system/myflaskapp.service)

```
[Unit]
Description=Gunicorn instance to serve Flask app
After=network.target

[Service]
User=www-data
Group=www-data
WorkingDirectory=/var/www/myflaskapp
Environment="PATH=/var/www/myflaskapp/venv/bin"

ExecStart=/var/www/myflaskapp/venv/bin/gunicorn \
    --workers 3 \
    --worker-class gthread \
    --threads 50 \
    --timeout 300 \
    --bind unix:/var/www/myflaskapp/myflaskapp.sock \
    testserver1:app \
    --access-logfile /var/log/myflaskapp/access.log \
    --error-logfile /var/log/myflaskapp/error.log

Restart=always

[Install]
WantedBy=multi-user.target
```

Commands to enable & start service:

```
sudo systemctl daemon-reload
sudo systemctl enable myflaskapp
sudo systemctl start myflaskapp
sudo systemctl status myflaskapp
```

4. Nginx Configuration (/etc/nginx/sites-available/myflaskapp)

```
server {
    listen 80;
    server_name 157.20.190.59;

    location / {
        include proxy_params;
        proxy_pass http://unix:/var/www/myflaskapp/myflaskapp.sock;
    }
}
sudo ln -s /etc/nginx/sites-available/myflaskapp /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

5. Testing

```
# Check Gunicorn (local socket)
curl -I http://127.0.0.1/home

# Check Nginx (public IP)
curl -I http://157.20.190.59/home
```

Expected Output: HTTP 200 OK

6. Notes for High Concurrency

- gthread worker class with `--threads 50` can handle ~150 concurrent users with 3 workers.
- Monitor logs in `/var/log/myflaskapp/`.
- Adjust threads & workers based on server RAM and CPU.

Let's calculate and reason this carefully. You're using **Gunicorn with gthread**:

- **Workers:** 3
 - **Threads per worker:** 50
 - **Total threads:** $3 \times 50 = 150$ threads
-

1. Memory Consideration

Memory consumption depends on:

1. **Python process base footprint** – A minimal Flask + Gunicorn worker with your dependencies (Flask, Firestore, Pandas, PyPDF2, etc.) will likely consume **50–70 MB per worker** as a starting point.
2. **Threads** – `gthread` threads share memory within the same process. So **threads do not multiply memory like separate processes**. Threads mainly use stack memory (~1 MB each by default).

Rough estimation:

Parameter	Value
Workers	3
Threads per worker	50
Base memory/worker	70 MB
Threads overhead	$50 \times 1 \text{ MB} = 50 \text{ MB}$
Total per worker	$70 + 50 = 120 \text{ MB}$
Total for 3 workers	$120 \times 3 \approx 360 \text{ MB}$

So you can expect **~350–400 MB RAM** for your current configuration, not counting Firestore responses in memory or other data-heavy operations.

2. Scaling Considerations

- **If traffic grows beyond 150 concurrent users**, you have a few options:
 1. **Increase threads per worker**
 - Pros: More concurrency per process without spawning more memory-heavy workers
 - Cons: Threads are limited by Python GIL for CPU-bound tasks, but since your app is mostly I/O (Firestore, PDF, emails), threads work well.
 2. **Increase number of workers**
 - Pros: Workers are separate processes → better CPU utilization, isolation, and fault tolerance

- Cons: Each worker duplicates memory footprint (~70 MB base + threads overhead)

3. Combination

- For example, 4 workers × 50 threads → total threads = 200
- Memory ~480 MB

Rule of thumb:

- For I/O-bound apps (like yours), **fewer workers + more threads** is usually sufficient.
- For CPU-heavy operations (PDF merges, Pandas heavy processing), **increase workers** for true parallelism.

1. Gunicorn Worker Types Comparison

Worker Type	How it works	Pros	Cons	When to use
sync	Each worker handles one request at a time . New requests wait in queue.	- Simple, stable - Good for CPU-bound apps	- Limited concurrency per worker - Queue builds under high traffic - Each worker = 1 CPU core ideally	CPU-bound apps or low traffic I/O apps
gthread	Each worker uses multiple threads to handle multiple requests simultaneously. Threads share memory.	- Good for I/O-bound apps - Handles many concurrent requests with fewer processes - Lower memory footprint than adding workers	- GIL limits true CPU parallelism - Each thread adds some memory overhead	I/O-bound apps like your Firestore/email/PDF app
gevent	Worker uses greenlets (cooperative coroutines) and monkey-patches I/O for async behavior.	- Handles many concurrent connections efficiently - Very low memory footprint	- Needs monkey-patching - Can break some libraries not compatible with gevent	High I/O apps where you can fully rely on async/non-blocking libraries

Worker Type	How it works	Pros	Cons	When to use
		per connection - Good for thousands of I/O-bound requests	- Debugging harder	

2. Your Case Analysis

- Your app is mostly **I/O-bound**:
 - Firestore reads/writes
 - Google Cloud Storage
 - Email via SMTP
 - File/PDF operations
- **CPU-heavy parts**: Pandas, PDF merging.
- You tried **gevent**, but it was timing out (probably Firestore + gRPC client issues).

✓ **Conclusion:** `gthread` is the best for your case because:

- It supports I/O-bound concurrency.
 - No monkey-patching needed (Firestore works fine).
 - Each thread can handle Firestore/email/PDF requests concurrently.
-

3. Scaling Plan for 500 Concurrent Users

Let's assume **500 concurrent requests** coming in.

Option 1: Using `gthread`

1. **Threads per worker**: 50
2. **Workers**: 10 (to handle peaks + CPU-intensive tasks)
3. **Total threads**: $10 \times 50 = 500$ threads

Memory Estimation:

- Base memory per worker: 70 MB
- Thread stack overhead: $50 \text{ threads} \times 1 \text{ MB} \approx 50 \text{ MB}$
- Memory per worker $\approx 120 \text{ MB}$
- Total memory for 10 workers $\approx 1.2 \text{ GB}$

CPU Requirements:

- Each worker can use 1 core effectively for Python + I/O
- Recommend **4–6 vCPUs** to avoid CPU saturation during PDF/Pandas operations

Gunicorn Config Example:

```
ExecStart=/var/www/myflaskapp/venv/bin/gunicorn \
--workers 10 \
--worker-class gthread \
--threads 50 \
--timeout 300 \
--bind unix:/var/www/myflaskapp/myflaskapp.sock \
testserver1:app
```

Advantages:

- Easy to scale threads/workers
- Memory is predictable
- Works with Firestore & other blocking I/O

Option 2: Using sync workers

- Each worker handles 1 request at a time.
- To handle 500 concurrent users: **500 workers needed** → impossible, huge RAM footprint.

✗ Not feasible.

Option 3: Using gevent workers

- Can handle 500+ connections with **5–10 workers**.
- Memory usage lower than gthread.
- Problem: Firestore Python client + gRPC sometimes blocks gevent → can cause timeout errors (you faced this).

✓ Only use gevent if all libraries are async-compatible.

4. Hardware Recommendation for 500 Concurrent Users

Resource Recommended for gthread (10 workers × 50 threads)

RAM 2 GB (1.2 GB app + buffer for OS and caching)

Resource Recommended for gthread (10 workers × 50 threads)

CPU 4–6 vCPU cores

Storage 20–50 GB (depends on PDF/temp files)

Network High bandwidth for Firestore & GCS

Notes:

- Increase RAM if you serve PDFs or large datasets.
 - CPU-intensive tasks (PDF merge, Pandas) may need more cores.
 - Use `htop` or `ps` to monitor memory under load.
-

5. Summary Recommendation

- **Use gthread:** 10 workers × 50 threads → 500 concurrent requests.
- **Timeout:** 300 seconds (for long I/O).
- **Memory:** ~1.2–1.5 GB, add buffer for OS and temp files.
- **CPU:** 4–6 cores for PDF/Pandas-heavy workloads.
- **Do not use gevent** with Firestore because of timeouts.
- **Sync workers** are impractical for 500 concurrent users.